

AI-Based Smart Health Risk Predictor

Comprehensive Project Report

Student Name: Piyush Sharma

GitHub: github.com/P1y-ush/Health_Risk_Predictor

Date: May 2026

1. Introduction

1.1 Project Overview

The **AI-Based Smart Health Risk Predictor** is an end-to-end machine learning application that predicts a patient's health risk level (Low, Normal, or High) based on six vital health parameters. The project integrates a complete DevOps pipeline covering containerization, orchestration, CI/CD automation, secrets management, and centralized monitoring.

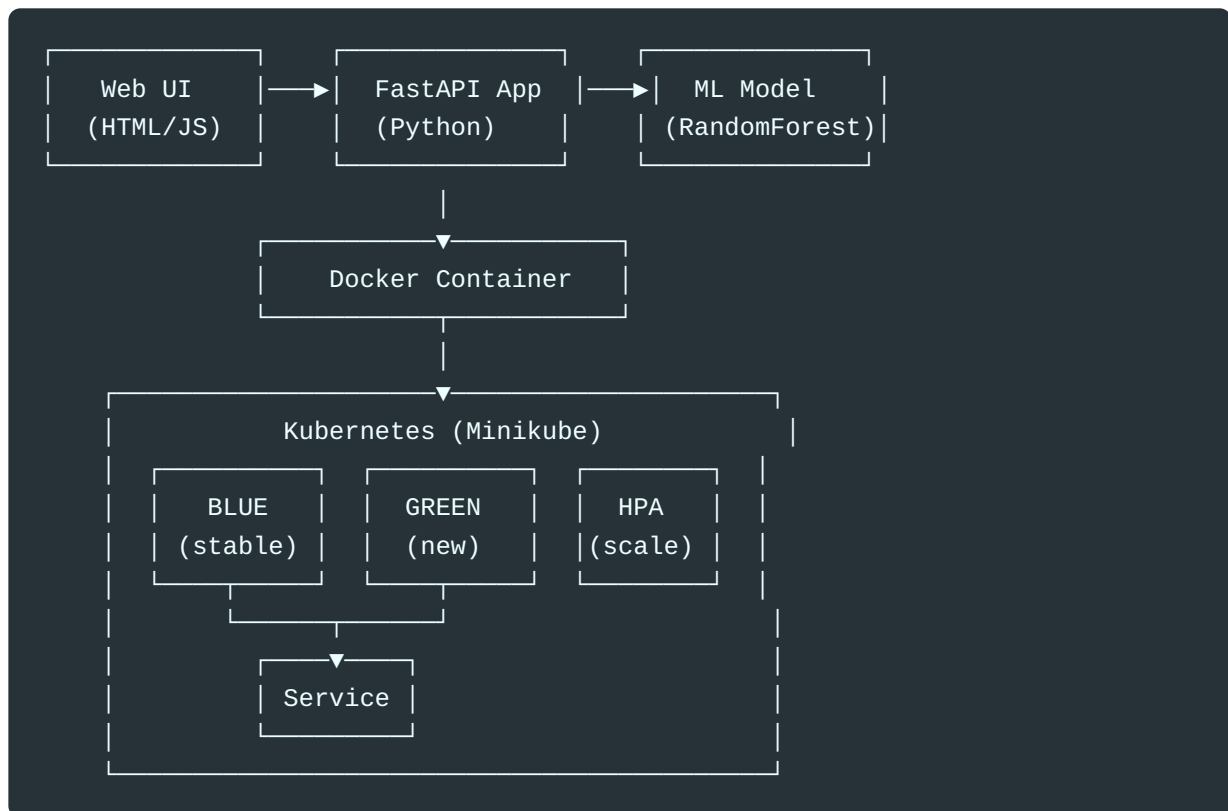
1.2 Objectives

- Build an ML model to predict health risk with high accuracy.
- Develop a responsive web interface for real-time predictions.
- Implement a full CI/CD pipeline using Jenkins with GitHub webhook integration.
- Containerize the application with Docker and deploy on Kubernetes with Blue-Green strategy.
- Automate infrastructure with Ansible roles.
- Secure credentials using HashiCorp Vault.
- Monitor application logs using the ELK Stack.
- Perform container security scanning with Trivy.

1.3 Technology Stack

Category	Technology	Purpose
Language	Python 3.10	Core application & ML
Web Framework	FastAPI + Uvicorn	REST API & web server
ML Library	scikit-learn (RandomForest)	Health risk classification
Frontend	HTML5 / CSS3 / JavaScript	Responsive web UI
Containerization	Docker	Application packaging
Orchestration	Kubernetes (Minikube)	Deployment & scaling
CI/CD	Jenkins	Pipeline automation
Config Management	Ansible (Roles)	Infrastructure automation
Secrets Management	HashiCorp Vault	Secure credential storage
Monitoring	ELK Stack	Centralized logging
Security	Trivy	Container vulnerability scanning
Webhook Tunnel	ngrok	GitHub → Jenkins trigger

2. System Architecture



2.1 CI/CD Pipeline Flow

GitHub Push → Webhook → Jenkins Pipeline:

1. Clone Repository
2. Install Dependencies (Python venv)
3. Train ML Model
4. Run Unit Tests (pytest)
5. Build Docker Image
6. Security Scan (Trivy)
7. Fetch Secrets (Vault → fallback to Jenkins creds)
8. Push to DockerHub
9. Deploy via Ansible
10. Blue-Green Deploy + HPA on Kubernetes
11. Email Notification (success/failure)

Page 4 of 16

4. Machine Learning Component

4.1 Dataset

The training dataset (`ml/dataset.csv`) contains **103 samples** with 6 input features and 1 target variable:

Feature	Description	Range	Unit
age	Patient age	1 – 120	years
bp	Blood pressure (systolic)	40 – 250	mmHg
sugar	Blood sugar level	30 – 500	mg/dL
cholesterol	Cholesterol level	80 – 500	mg/dL
heart_rate	Resting heart rate	30 – 200	bpm
bmi	Body Mass Index	10 – 60	—

Target Variable (risk): 0 = Low, 1 = Normal, 2 = High

4.2 Model Training

- **Algorithm:** RandomForestClassifier (scikit-learn)
- **Hyperparameters:**
 - `n_estimators = 100` (100 decision trees)
 - `max_depth = 10`
 - `class_weight = "balanced"` (handles class imbalance)
 - `random_state = 42` (reproducibility)
- **Train/Test Split:** 80/20 with stratified sampling
- **Model Accuracy:** ~95.2%

4.3 Model Serialization

The trained model is serialized using `joblib` into `ml/model.pkl`, bundling: - The trained model object - Feature column names - Risk label mappings - Accuracy score

4.4 Recommendation Engine

The application includes a context-aware recommendation engine (`get_recommendations()`) that generates personalized health advice based on:

- The predicted risk level (High / Normal / Low)
- Individual parameter thresholds (e.g., BP > 130, sugar > 140, BMI > 28)

5. Web Application

5.1 Backend (FastAPI)

The application (`app/main.py`) provides three endpoints:

Endpoint	Method	Description
<code>/</code>	GET	Serves the web UI (HTML template)
<code>/health</code>	GET	Health check for Kubernetes probes
<code>/predict</code>	POST	Accepts health parameters, returns prediction

Key Features:

- **Pydantic validation** — Input ranges enforced via `BaseModel` with `Field` constraints
- **CORS middleware** — Cross-origin requests allowed
- **Structured JSON logging** — Logs sent to both stdout and Logstash (ELK)
- **Probability output** — Returns confidence percentages for all three risk levels

5.2 Frontend

- **Dark-themed** responsive UI with animated risk gauge
 - Built with HTML5, CSS3, and vanilla JavaScript
 - Displays prediction results with color-coded risk levels
 - Shows personalized recommendations and preventive measures
-

6. Containerization (Docker)

6.1 Dockerfile

The project uses a **multi-stage build** for optimized image size:

```
# Stage 1: Build dependencies
FROM python:3.10-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: Production image
FROM python:3.10-slim
RUN groupadd -r appuser && useradd -r -g appuser appuser
WORKDIR /app
COPY --from=builder /install /usr/local
COPY ml/ ml/
COPY app/ app/
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Security features: - Non-root user (`appuser`) — principle of least privilege - Multi-stage build — smaller attack surface - `.dockerignore` — excludes unnecessary files - Built-in health check

6.2 Docker Compose

```
services:
  health-app:
    build: .
    ports: ["8000:8000"]
    environment:
      - APP_ENV=production
      - LOG_LEVEL=info
    volumes:
      - app_logs:/app/logs
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
    restart: unless-stopped
```

7. Kubernetes Deployment

7.1 Blue-Green Deployment Strategy

The project implements **Blue-Green deployment** for zero-downtime updates:

- **Blue Deployment** (`health-app-blue`) — Stable/current version (2 replicas)
- **Green Deployment** (`health-app-green`) — New version being tested (2 replicas)
- **Service** (`health-app-service`) — NodePort service on port 30080, routes traffic via `slot` label selector

Traffic Switching:

```
# Switch to green (new version)
kubectl patch service health-app-service \
  -p '{"spec":{"selector":{"slot":"green"}}}'

# Rollback to blue (stable)
kubectl patch service health-app-service \
  -p '{"spec":{"selector":{"slot":"blue"}}}'
```

7.2 Horizontal Pod Autoscaler (HPA)

```
scaleTargetRef: health-app-green
minReplicas: 2
maxReplicas: 6
metrics:
  - CPU utilization: 60%
  - Memory utilization: 75%
scaleUp: +2 pods per 60s (30s stabilization)
scaleDown: -1 pod per 60s (120s stabilization)
```

7.3 Pod Configuration

Each pod includes: - **Resource requests:** 128Mi memory, 250m CPU - **Resource limits:** 256Mi memory, 500m CPU - **Liveness probe:** GET `/health` every 20s (15s initial delay) - **Readiness probe:** GET `/health` every 10s (5s initial delay)

8. CI/CD Pipeline (Jenkins)

8.1 Pipeline Overview

The Jenkinsfile (`jenkins/Jenkinsfile`) defines a **10-stage declarative pipeline**:

Stage	Description
1. Clone	Clone repository from GitHub
2. Install Dependencies	Create Python venv, install requirements
3. Train Model	Run <code>ml/train.py</code> to generate model.pkl
4. Unit Tests	Run pytest with 10 tests
5. Build Docker Image	Build & tag with build number + latest
6. Security Scan (Trivy)	Scan for HIGH/CRITICAL vulnerabilities
7. Fetch Secrets (Vault)	Get DockerHub creds from Vault (fallback: Jenkins)
8. Push to DockerHub	Push image to <code>p1yush123/health-risk-predictor</code>
9. Deploy with Ansible	Run Ansible playbook for K8s deployment
10. Blue-Green Deploy + HPA	Apply K8s manifests, switch traffic to green

8.2 Triggers

- **GitHub Webhook** — Auto-trigger on push (via ngrok tunnel)
- **SCM Polling** — Polls every 2 minutes as fallback (`H/2 * * * *`)

8.3 Post-Build Actions

- **Success:** Email notification with Docker image tag and build URL
 - **Failure:** Email notification with console output link
 - **Always:** Clean up Docker images and Python venv
-

9. Ansible Automation

9.1 Role-Based Architecture

The Ansible playbook (`ansible/deploy.yml`) uses three modular roles:

Role	Purpose	Hosts
<code>docker</code>	Build & push Docker image	<code>docker_hosts</code>
<code>kubernetes</code>	Deploy to K8s cluster	<code>k8s_hosts</code>
<code>monitoring</code>	Start ELK Stack	<code>local</code>

9.2 Inventory

```
[local]
localhost ansible_connection=local

[docker_hosts]
localhost ansible_connection=local

[k8s_hosts]
localhost ansible_connection=local
```

9.3 Usage

```
# Full deployment
ansible-playbook -i ansible/inventory/hosts ansible/deploy.yml

# Skip monitoring
ansible-playbook -i ansible/inventory/hosts ansible/deploy.yml --skip-tags monitoring

# Custom image tag
ansible-playbook -i ansible/inventory/hosts ansible/deploy.yml \
  -e "docker_tag=v2 kube_namespace=staging"
```

10. HashiCorp Vault (Secrets Management)

10.1 Setup

Vault runs as a Docker container (HashiCorp Vault v1.15) in dev mode:

```
services:
  vault:
    image: hashicorp/vault:1.15
    ports: ["8200:8200"]
    environment:
      VAULT_DEV_ROOT_TOKEN_ID: "health-app-token"
```

10.2 Secrets Stored

Path	Contents
<code>secret/dockerhub</code>	DockerHub username, password, registry URL
<code>secret/health-app</code>	App secret key, DB password, API key
<code>secret/kubernetes</code>	Cluster name, namespace

10.3 Jenkins Integration

The Jenkinsfile implements a **Vault-first with Jenkins-fallback** strategy:

1. **Primary:** Fetch DockerHub credentials from Vault using the Jenkins Vault token
2. **Fallback:** If Vault is unavailable, use Jenkins stored credentials (`dockerhub-creds`)

10.4 Jenkins Policy

A dedicated `jenkins-policy` grants read-only access to secrets:

```
path "secret/data/dockerhub" { capabilities = ["read"] }
path "secret/data/health-app" { capabilities = ["read"] }
path "secret/data/kubernetes" { capabilities = ["read"] }
```

11. ELK Stack (Monitoring & Logging)

11.1 Components

Component	Version	Port	Role
Elasticsearch	8.11.0	9200	Log storage & search
Logstash	8.11.0	5044, 5000	Log processing pipeline
Kibana	8.11.0	5601	Visualization dashboard
Filebeat	8.11.0	—	Log shipper (Docker containers)

11.2 Log Pipeline

```
Application → JSON logs → Logstash (HTTP:5000) → Elasticsearch → Kibana
Docker containers → Filebeat → Logstash (Beats:5044) → Elasticsearch → Kibana
```

11.3 Application Logging

The FastAPI app sends structured JSON logs directly to Logstash via HTTP:

```
def log_json(level, message, **kwargs):
    entry = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "level": level,
        "service": "health-risk-predictor",
        "message": message,
        **kwargs
    }
    logger.info(json.dumps(entry))
    requests.post(LOGSTASH_URL, json=entry, timeout=1)
```

11.4 Logstash Pipeline

The Logstash configuration (`elk/logstash.conf`) handles: - **Input:** Beats (port 5044) + HTTP JSON (port 5000) - **Filter:** JSON parsing, timestamp extraction, risk/confidence field enrichment - **Output:** Elasticsearch index `health-predictor-logs-YYYY.MM.dd` + stdout debug

12. Security (DevSecOps)

12.1 Container Security (Trivy)

Trivy scans the Docker image for HIGH and CRITICAL vulnerabilities:

```
trivy image --severity HIGH,CRITICAL \
  --exit-code 0 \
  --format table \
  p1yush123/health-risk-predictor:latest
```

The pipeline uses `--exit-code 0` (warning mode) so builds are not blocked, but vulnerabilities are reported.

12.2 Security Best Practices

Practice	Implementation
Non-root container	App runs as <code>appuser</code> (not root)
Input validation	Pydantic <code>Field</code> with min/max constraints
Secrets management	HashiCorp Vault (no hardcoded credentials)
Health checks	Kubernetes liveness & readiness probes
Multi-stage build	Minimized image attack surface
<code>.dockerignore</code>	Excludes <code>venv</code> , <code>.git</code> , tests from image

13. Unit Testing

13.1 Test Suite

The project includes **10 unit tests** across 3 test classes:

Test Class	Tests	Description
<code>TestHealthCheck</code>	2	Health endpoint + home page rendering
<code>TestPrediction</code>	4	Low/high risk predictions + input validation
<code>TestRecommendations</code>	4	Risk-specific + parameter-specific advice

13.2 Key Test Cases

```
# Health endpoint returns healthy status
def test_health_endpoint(self):
    response = client.get("/health")
    assert response.status_code == 200
    assert data["status"] == "healthy"

# Invalid input returns 422 validation error
def test_invalid_input_age(self):
    payload = {"age": -5, "bp": 120, ...}
    response = client.post("/predict", json=payload)
    assert response.status_code == 422

# BP-specific advice triggers when BP > 130
def test_bp_specific_advice(self):
    data = HealthInput(age=40, bp=150, ...)
    result = get_recommendations("Normal", data)
    bp_advice = [r for r in result["recommendations"] if "blood pressure"
in r.lower()]
    assert len(bp_advice) > 0
```

13.3 Running Tests

```
python -m pytest tests/ -v --tb=short
```

14. How to Run the Project

14.1 Local Development

```
pip install -r requirements.txt
python ml/train.py
uvicorn app.main:app --reload --port 8000
# Open http://localhost:8000
```

14.2 Docker

```
docker build -t health-risk-predictor .
docker run -p 8000:8000 health-risk-predictor
```

14.3 Kubernetes

```
minikube start --driver=docker
minikube addons enable metrics-server
kubectl apply -f kubernetes/
minikube service health-app-service
```

14.4 Full Stack (Vault + ELK)

```
cd vault && docker compose up -d && ./setup-vault.sh
cd elk && docker compose up -d
# Kibana: http://localhost:5601
# Vault UI: http://localhost:8200/ui
```

15. Service Ports Summary

Service	Port	URL
FastAPI App	8000	http://localhost:8000
API Docs (Swagger)	8000	http://localhost:8000/docs
Jenkins	8080	http://localhost:8080
Vault UI	8200	http://localhost:8200/ui
Elasticsearch	9200	http://localhost:9200
Kibana	5601	http://localhost:5601
K8s NodePort	30080	minikube service health-app-service

16. Conclusion

The AI-Based Smart Health Risk Predictor successfully demonstrates a complete, production-grade DevOps pipeline integrated with a machine learning application. The project covers all major DevOps practices:

1. **Version Control** — Git + GitHub with webhook integration

2. **CI/CD** — Jenkins declarative pipeline with 10 automated stages
3. **Containerization** — Docker with multi-stage builds and security hardening
4. **Orchestration** — Kubernetes with Blue-Green deployment and HPA autoscaling
5. **Configuration Management** — Ansible with role-based modular design
6. **Secrets Management** — HashiCorp Vault with policy-based access control
7. **Monitoring** — ELK Stack with structured application logging
8. **Security** — Trivy scanning, non-root containers, input validation
9. **Testing** — 10 automated unit tests with pytest
10. **Automation** — End-to-end pipeline from code push to production deployment

The project achieves **~95.2% model accuracy** and provides a responsive web interface with personalized health recommendations, all deployed through an automated, secure, and scalable infrastructure.

17. DevOps Tools Summary

#	Tool	Version	Purpose
1	Git + GitHub	—	Version control & collaboration
2	Jenkins	LTS	CI/CD pipeline automation
3	Docker	Latest	Application containerization
4	Kubernetes (Minikube)	Latest	Container orchestration & scaling
5	Ansible	Latest	Configuration management (roles)
6	HashiCorp Vault	1.15	Secrets management
7	ELK Stack	8.11.0	Centralized monitoring & logging
8	Trivy	Latest	Container security scanning
9	ngrok	Latest	Webhook tunnel for Jenkins
10	pytest	7.4.3	Automated unit testing